TRƯỜNG ĐẠI HỌC BÁCH KHOA – ĐẠI HỌC ĐÀ NẴNG
KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO

ĐỀ TÀI

Thiết kế giao diện đa màn hình: Responsive & Adaptive UI trong Flutter

Sinh viên thực hiện: **Phan Văn Trà**
Trần Đăng Duy

Giảng viên hướng dẫn: **TS. Nguyễn Duy Nhật Viễn**

Mục lục

1. Cơ sở lý thuyết

1.1 Khái niệm Responsive và Adaptive

1.2 MediaQuery, LayoutBuilder và breakpoint

1.3 Nguyên tắc thiết kế cho nhiều màn hình và phương thức nhập liệu

2. Ứng dụng Adaptive Notes

2.1 Tổng quan ứng dụng Adaptive Notes

2.2 Bố cục thích ứng và minh hoạ mã nguồn

2.3 Demo sản phẩm

3. Kết luận

Phân công công việc

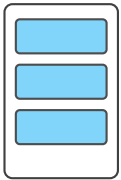
Thành viên	Nội dung công việc	Tỷ lệ
Phan Văn Trà	• Tìm hiểu và phân biệt Responsive UI và Adaptive UI. • LayoutBuilder và breakpoint theo không gian khả dụng. • Xây dựng, minh họa hai bố cục theo chiều rộng.	**50%**
Trần Đăng Duy	• MediaQuery: kích thước, hướng hiển thị và cỡ chữ. • Nguyên tắc thiết kế đa kích thước và phương thức nhập liệu. • Lưới co giãn; hỗ trợ cảm ứng, chuột và bàn phím.	**50%**

1.1 Responsive và Adaptive: hai cách thích ứng giao diện

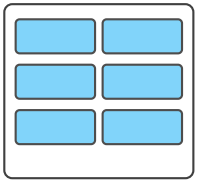
Tiêu chí	Responsive	Adaptive
Ý tưởng	Điều chỉnh kích thước và sắp xếp theo không gian khả dụng	**Chọn** một trong nhiều bố cục riêng theo điều kiện
Công cụ	**`Expanded`**, **`Flexible`**, **`GridView`**, **`ConstrainedBox`**	**`LayoutBuilder`** / **`MediaQuery`** + **breakpoint**, trả về widget khác nhau
Ví dụ	Lưới thẻ tự chia **1** → **2** → **3 cột**	Danh sách (mobile) ↔ **Master–Detail** (tablet)
Phù hợp khi	Kích thước thay đổi liên tục (kéo cửa sổ, xoay màn hình)	Cấu trúc giao diện hoặc cách tương tác khác nhau giữa các nhóm màn hình

Thực tế thường **kết hợp cả hai**: **adaptive** chọn bố cục khung, **responsive** co giãn nội dung bên trong.

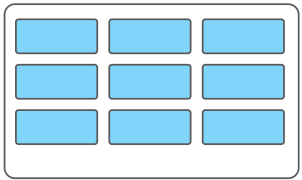
1.1 Minh họa: giao diện đáp ứng và thích ứng



hẹp: **1 cột**

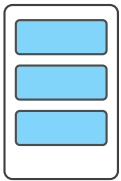


vừa: **2 cột**

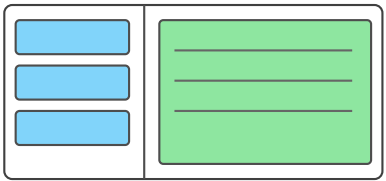


rộng: **3 cột**

Responsive
Điều chỉnh kích thước và số cột theo không gian khả dụng.



Mobile: danh sách



Tablet: danh sách | chi tiết

Adaptive
Thay đổi cấu trúc bố cục: một vùng hoặc hai vùng.

1.2 MediaQuery: thông tin về môi trường hiển thị

• MediaQuery cung cấp thông tin về cửa sổ ứng dụng.

• Các thuộc tính thường dùng: size, orientation, padding, textScaler và platformBrightness.

• Các hàm sizeOf, orientationOf và textScalerOf chỉ tạo phụ thuộc vào thuộc tính tương ứng.

• Kích thước cửa sổ không biểu thị ràng buộc bố cục của từng widget con.

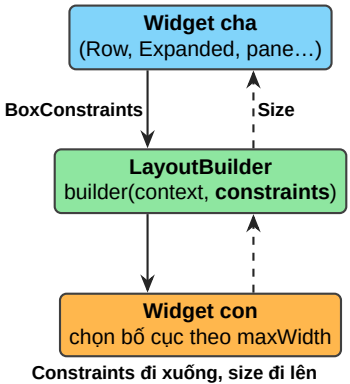
```dart
// Kích thước cửa sổ / màn hình (dp)
final size = MediaQuery.sizeOf(context);
final width = size.width;

// Hướng màn hình
final orientation = MediaQuery.orientationOf(context);
final isLandscape =
    orientation == Orientation.landscape;

// Cỡ chữ người dùng chọn trong hệ thống
final scaler = MediaQuery.textScalerOf(context);

// Vùng an toàn (tai thỏ, thanh trạng thái)
final padding = MediaQuery.paddingOf(context);
```

1.2 LayoutBuilder: ràng buộc bố cục từ widget cha




```
LayoutBuilder(
  builder: (context, constraints) {
    // maxWidth: bề rộng tối đa khả dụng cho widget này
    if (constraints.maxWidth >= 600) {
      return const TabletLayout();
    }
    return const MobileLayout();
  },
)
```

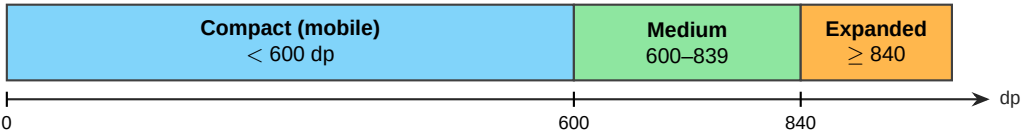
- builder được gọi lại khi ràng buộc bố cục thay đổi.
- Phù hợp với từng vùng hiển thị hoặc cửa sổ ứng dụng có thể thay đổi kích thước.

1.2 So sánh MediaQuery và LayoutBuilder

Tiêu chí	MediaQuery	LayoutBuilder
Thông tin cung cấp	**Cửa sổ / màn hình** và thông tin hệ thống	**Ràng buộc** (BoxConstraints) mà widget cha cấp
Dùng khi	Cần cỡ chữ, vùng an toàn, hướng, sáng/tối	Chọn bố cục theo **không gian thực sự khả dụng**
Hạn chế	Không xác định ràng buộc bố cục của từng vùng con trong cửa sổ.	Không cung cấp thông tin hệ thống như cỡ chữ hoặc chế độ sáng/tối.
Khuyến nghị	Dùng cho **thông tin hệ thống**	Dùng để **chọn bố cục**

MediaQuery vẫn cập nhật kích thước cửa sổ khi chạy chia đôi màn hình.

1.2 Breakpoint: ngưỡng để đổi bố cục



- **Breakpoint** là giá trị bề rộng (đơn vị **dp**) mà tại đó bố cục thay đổi.
- Dựa vào **bề rộng khả dụng**, không dựa vào tên thiết bị.
- Dùng **ít** breakpoint, đặt thành **hằng số** có tên.
- Mốc trên theo phân loại kích thước cửa sổ của **Material 3**.

```
enum ScreenType { mobile, tablet }

class Breakpoints {
  static const double tablet = 600;

  static ScreenType fromWidth(double width) =>
      width >= tablet
          ? ScreenType.tablet
          : ScreenType.mobile;
}
```

Ứng dụng: 600dp là ngưỡng đổi bố cục.
400/900dp là các ngưỡng đổi số cột.
900dp là giá trị tùy chỉnh của ứng dụng.

1.3 Nguyên tắc thiết kế giao diện thích ứng

1. Theo không gian khả dụng
Dùng LayoutBuilder; không suy luận bố cục từ tên thiết bị hoặc chỉ từ hướng xoay.

2. Giới hạn chiều rộng
Giới hạn chiều rộng nội dung và biểu mẫu để bảo đảm khả năng đọc.

3. **Bảo toàn trạng thái**
Đổi kích thước hoặc xoay **không làm mất** lựa chọn và dữ liệu đang xem.

4. Đa phương thức nhập liệu
Hỗ trợ thao tác cảm ứng, chuột và bàn phím phù hợp với chức năng.

5. Khả năng tiếp cận
Hỗ trợ cỡ chữ hệ thống, chế độ sáng/tối và vùng chạm đủ lớn.

6. Hạn chế tràn bố cục
Sử dụng Expanded, Flexible và vùng cuộn; kiểm thử ở nhiều kích thước.

1.3 Nhập liệu bằng cảm ứng, chuột và bàn phím

Phương thức	Đặc trưng	Cách thích ứng trong Flutter
Cảm ứng	Ngón tay, không có hover, cần vùng chạm lớn	Vùng chạm đủ lớn; cử chỉ **vuốt** (**`Dismissible`**), **`GestureDetector`**
Chuột	Con trỏ chính xác, có **hover**, chuột phải	**`InkWell`**, **`MouseRegion`**, **`mouseCursor`**; nút bấm **hiển thị sẵn** thay cho cử chỉ ẩn
Bàn phím	Phím tắt, di chuyển bằng **focus**	**`Shortcuts`** / **`Actions`**, **`CallbackShortcuts`**, **`Focus`**

2.1 Adaptive Notes – ứng dụng ghi chú đa màn hình

• Minh họa Responsive và Adaptive bằng ứng dụng ghi chú sử dụng thư viện Flutter.

- **2 bố cục**: **Mobile** ($<$ 600 dp, 1 vùng) và **Tablet** ($\geq$ 600 dp, **Master–Detail**).
- Lưới thẻ chia **1 / 2 / 3 cột** theo bề rộng thực của vùng danh sách.

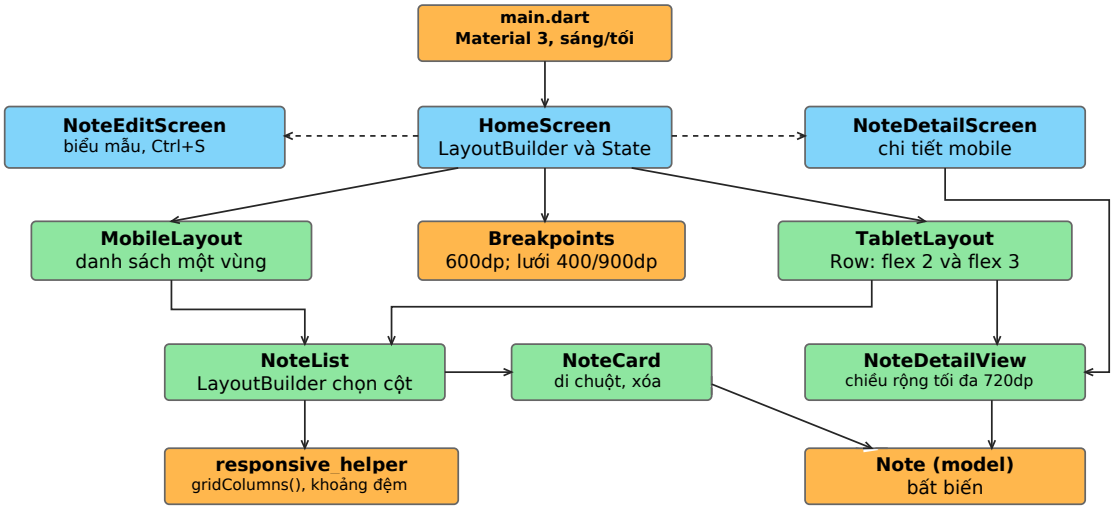
• Hỗ trợ xem, thêm, chỉnh sửa, xóa ghi chú và hoàn tác thao tác xóa bằng SnackBar.

• Hỗ trợ cảm ứng, di chuột và các phím tắt Ctrl+N, Ctrl+S.

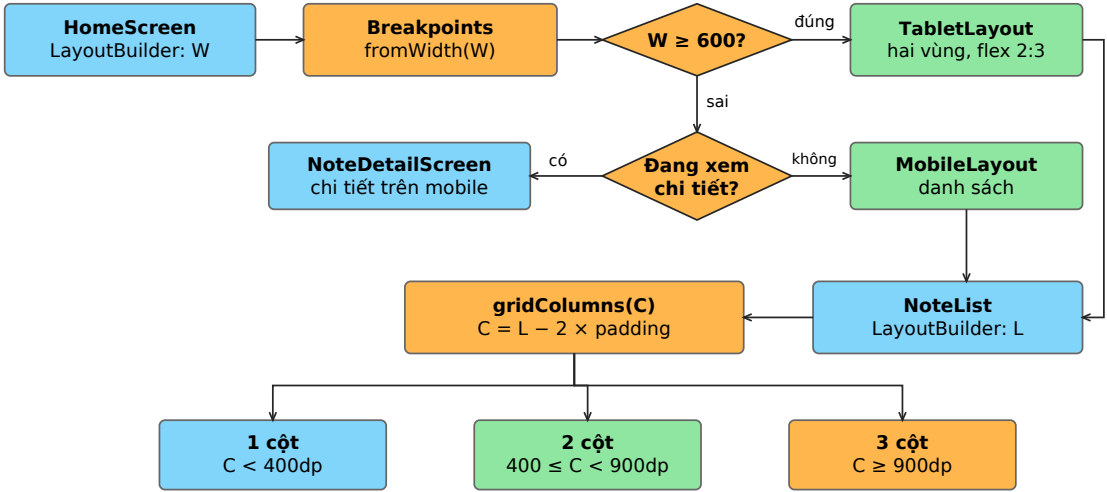
• Hỗ trợ chế độ sáng/tối, cỡ chữ hệ thống và thanh thông số bố cục.

Thành phần	Chi tiết
Ngôn ngữ	Dart
Framework	Flutter, Material 3
Công cụ	**`LayoutBuilder`**, **`MediaQuery`**, **`Shortcuts`**
Dữ liệu	In-memory, 10 ghi chú mẫu
Kiểm thử	**`flutter_test`**

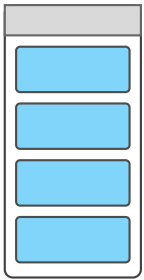
2.1 Kiến trúc ứng dụng (thư mục lib/)



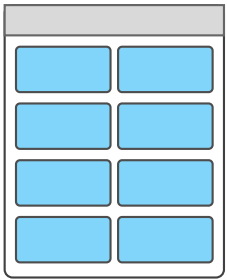
2.2 Quy trình lựa chọn bố cục của ứng dụng



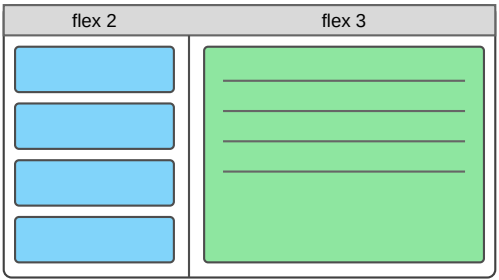
2.2 Minh hoạ bố cục theo không gian khả dụng



Mobile hẹp
Nội dung <400dp
1 cột



Mobile rộng
Nội dung ≥400dp
2 cột



Tablet: vùng bố cục ≥600dp
Master–Detail: danh sách và chi tiết
Số cột tính riêng trong pane danh sách

Ví dụ mobile: danh sách rộng 424dp − 12dp × 2 = 400dp nội dung, đủ 2 cột.
Ngưỡng 600dp chọn bố cục; ngưỡng 400/900dp chọn số cột theo nội dung.

2.2 Mã nguồn (1/5): breakpoint theo không gian

Trích trong class Breakpoints

```
static const double tablet = 600;

static ScreenType fromWidth(double width) =>
    width >= tablet
        ? ScreenType.tablet
        : ScreenType.mobile;
```

Ngưỡng lưới trong Breakpoints

```
static const double gridTwoColumns = 400;
static const double gridThreeColumns = 900;
```

Bố cục thích ứng

600dp là ngưỡng chuyển giữa một vùng và hai vùng.

Lưới co giãn

400/900dp áp dụng riêng cho nội dung vùng danh sách.

Theo bề rộng

Cùng một thiết bị có thể dùng nhiều bố cục khác nhau.

2.2 Mã nguồn (2/5): LayoutBuilder chọn bố cục

Trong builder của LayoutBuilder — HomeScreen

```
final type = Breakpoints.fromWidth(
    constraints.maxWidth);
```

Nhánh widget trong body — trích logic chọn bố cục

```
child: type == ScreenType.tablet
    ? TabletLayout(...)
    : detailVisible
        ? NoteDetailScreen(...)
        : MobileLayout(...),
```

detailVisible: đang xem chi tiết trên bố cục hẹp.

Đo vùng thực tế

constraints.maxWidth là bề rộng widget cha cấp.

Thay đổi bố cục

Rộng: danh sách và chi tiết.
Hẹp: xem lần lượt.

Phạm vi trích

Dấu ... biểu thị các đối số dữ liệu và hàm xử lý đã lược bỏ.

2.2 Mã nguồn (3/5): lưới theo vùng danh sách

responsive_helper.dart — chọn số cột

```
int gridColumns(double availableWidth) {
  if (availableWidth >=
      Breakpoints.gridThreeColumns) return 3;
  if (availableWidth >=
      Breakpoints.gridTwoColumns) return 2;
  return 1;
}
```

Trong NoteList — đo vùng và áp dụng số cột

```
final columns = gridColumns(
    constraints.maxWidth - padding * 2);

crossAxisCount: columns,
```

1 cột

Nội dung <400dp

2 cột

400dp ≤ nội dung <900dp

3 cột

Nội dung ≥900dp

LayoutBuilder đo riêng pane danh sách. Padding được trừ ở cả hai bên.

2.2 Mã nguồn (4/5): cỡ chữ và bề rộng dễ đọc

Trong NoteList.build — đọc cỡ chữ hệ thống

```
final scaler =
    MediaQuery.textScalerOf(context);
```

Thuộc tính của SliverGridDelegate

```
mainAxisExtent:
    100 + scaler.scale(17) * 3
        + scaler.scale(14) * 6,
```

Biểu mẫu: thuộc tính ConstrainedBox

```
constraints: const BoxConstraints(maxWidth: 640),
```

Hỗ trợ cỡ chữ hệ thống

MediaQuery đọc mức phóng chữ người dùng chọn.

Thẻ cao hơn

Khi chữ lớn, chiều cao thẻ tăng để có chỗ hiển thị.

Giới hạn chiều rộng

Biểu mẫu: tối đa 640dp.
Chi tiết: tối đa 720dp.

2.2 Mã nguồn (5/5): nhập liệu bằng bàn phím

bindings của CallbackShortcuts — HomeScreen

```
bindings: {
  const SingleActivator(
    LogicalKeyboardKey.keyN, control: true,
  ): () => _edit(),
},
```

bindings của CallbackShortcuts — NoteEditScreen

```
bindings: {
  const SingleActivator(
    LogicalKeyboardKey.keyS, control: true,
  ): _save,
},
```

Ctrl+N: thêm mới

Gọi _edit() để mở biểu mẫu tạo ghi chú.

Ctrl+S: lưu

Gọi _save() trong màn hình chỉnh sửa ghi chú.

Chạm và chuột

Hỗ trợ chạm, di chuột, vuốt sang trái và nút xóa.

2.3 Demo sản phẩm: điện thoại dọc, 393dp



393dp < 600dp: bố cục MOBILE

Thanh thông số xác nhận hướng dọc và bề rộng khả dụng.

Danh sách một vùng

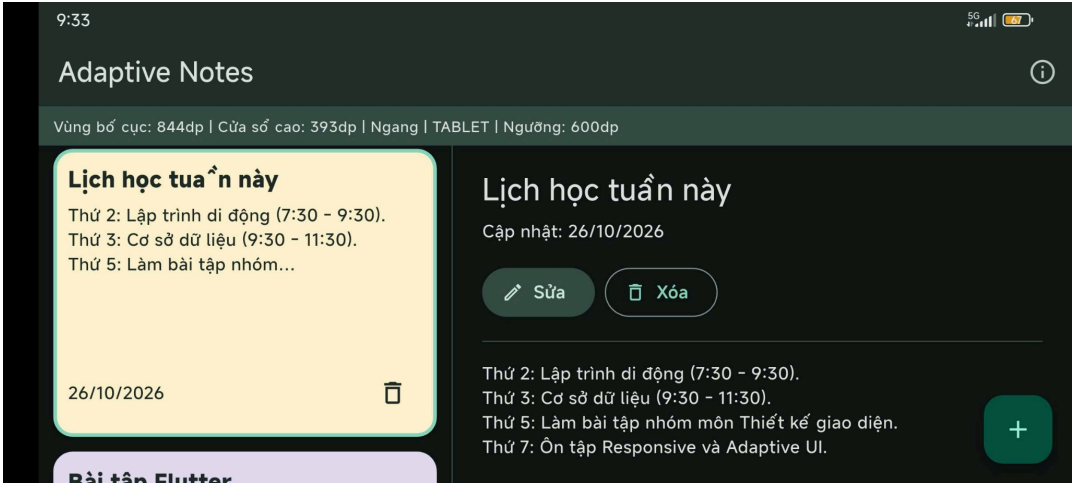
Các ghi chú xếp một cột. Người dùng chọn một ghi chú để mở chi tiết.

Ảnh giao diện thực tế

Nút + dùng để tạo ghi chú; biểu tượng thùng rác dùng để xóa. Ảnh chụp ở chế độ tối.

Mục tiêu trình diễn: xác nhận bố cục thay đổi theo không gian khả dụng.

2.3 Demo sản phẩm: điện thoại ngang, 844dp



844dp ≥ 600dp: TABLET. Danh sách bên trái, chi tiết bên phải trên cùng điện thoại.

3. Kết luận

• Responsive điều chỉnh kích thước và cách sắp xếp theo không gian khả dụng. Adaptive lựa chọn cấu trúc bố cục phù hợp với điều kiện hiển thị.

• LayoutBuilder cung cấp ràng buộc bố cục từ widget cha. MediaQuery cung cấp thông tin cửa sổ và thiết lập hiển thị của hệ thống.

• Khai báo tập trung các breakpoint, giới hạn chiều rộng nội dung và duy trì trạng thái khi thay đổi kích thước cửa sổ.

• Adaptive Notes minh họa hai bố cục và nhiều phương thức nhập liệu.

Xin cảm ơn
thầy và các bạn đã lắng nghe!

Hỏi & Đáp